



FMSV Project

Kovid Juneja-220101059

PROBLEM - 1

*APPLICATIONS TO MUTUAL
EXCLUSION*

LINGELING SAT SOLVER

HERE IS A BRIEF EXPLANATION OF THE QUESTION.



Alice and Bob are casual friends who share an apartment. One of their joint rooms is special: When they're in that critical room, which has two doors, they don't want the other person to be present. Furthermore, being busy people, they don't want to interrupt each other needlessly. So they agree to control access to the room by using an indicator light, which can be switched on or off.

A0. Maybe go to A1.

A1. If l go to A1, else to A2.

A2. Set $l \leftarrow 1$, go to A3.

A3. Critical, go to A4.

A4. Set $l \leftarrow 0$, go to A0.

B0. Maybe go to B1.

B1. If l go to B1, else to B2.

B2. Set $l \leftarrow 1$, go to B3.

B3. Critical, go to B4.

B4. Set $l \leftarrow 0$, go to B0.

(40)

PROBLEM FACED

Unfortunately, Alice and Bob soon learned that protocol (40) is unreliable:
One day she went from A1 to A2 and he went from B1 to B2,
before either of them had switched the indicator on.
Embarrassment (A3 and B3) followed.

They could have discovered this problem in advance, if they'd converted
the state transitions of (40) into clauses for bounded model checking,
as below, then applied a SAT solver.

CLAUSES

$$A0_0 \wedge \overline{A1_0} \wedge \overline{A2_0} \wedge \overline{A3_0} \wedge \overline{A4_0} \wedge B0_0 \wedge \overline{B1_0} \wedge \overline{B2_0} \wedge \overline{B3_0} \wedge \overline{B4_0} \wedge \overline{l_0}, \quad (41)$$

the following clauses for $0 \leq t < r$ (discussed in exercise 87) will emulate the first r steps of every legitimate scenario defined by (40):

$$\begin{array}{lll}
 (@_t \vee \overline{A0_t} \vee A0_{t+1}) & (\overline{@_t} \vee \overline{A0_t} \vee A0_{t+1} \vee A1_{t+1}) & (@_t \vee \overline{B0_t} \vee B0_{t+1} \vee B1_{t+1}) \\
 (@_t \vee \overline{A1_t} \vee A1_{t+1}) & (\overline{@_t} \vee \overline{A1_t} \vee \overline{l_t} \vee A1_{t+1}) & (@_t \vee \overline{B1_t} \vee \overline{l_t} \vee B1_{t+1}) \\
 (@_t \vee \overline{A2_t} \vee A2_{t+1}) & (\overline{@_t} \vee \overline{A1_t} \vee l_t \vee A2_{t+1}) & (@_t \vee \overline{B1_t} \vee l_t \vee B2_{t+1}) \\
 (@_t \vee \overline{A3_t} \vee A3_{t+1}) & (\overline{@_t} \vee \overline{A2_t} \vee A3_{t+1}) & (@_t \vee \overline{B2_t} \vee B3_{t+1}) \\
 (@_t \vee \overline{A4_t} \vee A4_{t+1}) & (\overline{@_t} \vee \overline{A2_t} \vee l_{t+1}) & (@_t \vee \overline{B2_t} \vee l_{t+1}) \\
 (\overline{@_t} \vee \overline{B0_t} \vee B0_{t+1}) & (\overline{@_t} \vee \overline{A3_t} \vee A4_{t+1}) & (@_t \vee \overline{B3_t} \vee B4_{t+1}) \\
 (\overline{@_t} \vee \overline{B1_t} \vee B1_{t+1}) & (\overline{@_t} \vee \overline{A4_t} \vee A0_{t+1}) & (@_t \vee \overline{B4_t} \vee B0_{t+1}) \\
 (\overline{@_t} \vee \overline{B2_t} \vee B2_{t+1}) & (\overline{@_t} \vee \overline{A4_t} \vee \overline{l_{t+1}}) & (@_t \vee \overline{B4_t} \vee \overline{l_{t+1}}) \\
 (\overline{@_t} \vee \overline{B3_t} \vee B3_{t+1}) & (\overline{@_t} \vee l_t \vee A2_t \vee A4_t \vee \overline{l_{t+1}}) & (@_t \vee l_t \vee B2_t \vee B4_t \vee \overline{l_{t+1}}) \\
 (@_t \vee \overline{B4_t} \vee B4_{t+1}) & (@_t \vee \overline{l_t} \vee A2_t \vee A4_t \vee l_{t+1}) & (@_t \vee \overline{l_t} \vee B2_t \vee B4_t \vee l_{t+1})
 \end{array} \quad (42)$$

EXPLANATION OF VARIABLES

In the example there are eleven variables $A0t, A1t, A2t, A3t, A4t, B0t, B1t, B2t, B3t, B4t, It$, together with ten binary exclusion clauses $(A0t \vee A1t), (A0t \vee A2t), \dots, (A3t \vee A4t)$ to ensure that Alice is in at most one state, and with ten similar clauses for Bob.

There's also a variable $@t$ which is true or false depending on whether Alice or Bob executes their program step at time t . (We say that Alice was "bumped" if $@t = 1$, and Bob was bumped if $@t = 0$.)

To improve the above problem we introduced new variables a, b such that Alice and Bob have more control on who enters as below

A0. Maybe go to A1.
A1. Set $a \leftarrow 1$, go to A2.
A2. If b go to A2, else to A3.
A3. Critical, go to A4.
A4. Set $a \leftarrow 0$, go to A0.

B0. Maybe go to B1.
B1. Set $b \leftarrow 1$, go to B2.
B2. If a go to B2, else to B3.
B3. Critical, go to B4.
B4. Set $b \leftarrow 0$, go to B0.

DEAD LOCK

- However, a new problem now arises, namely the problem known as “deadlock” or “livelock.”
- Alice and Bob can get into states A2 and B2, after which they’re stuck—each waiting for the other to go critical.
- To address the above deadlock problem, we introduced one new statements for Alice and Bob, ensuring that deadlock is no longer possible.

A0. Maybe go to A1.

A1. Set $a \leftarrow 1$, go to A2.

A2. If b go to A3, else to A4.

A3. Set $a \leftarrow 0$, go to A1.

A4. Critical, go to A5.

A5. Set $a \leftarrow 0$, go to A0.

B0. Maybe go to B1.

B1. Set $b \leftarrow 1$, go to B2.

B2. If a go to B3, else to B4.

B3. Set $b \leftarrow 0$, go to B1.

B4. Critical, go to B5.

B5. Set $b \leftarrow 0$, go to B0.

STARVATION

- But he realized that this too is unsatisfactory, because it permits scenarios in which Alice, say, might **wait forever** while Bob repeatedly uses the critical room. (Indeed, if Alice and Bob are in states A1 and B2, she might go to A2, A3, then A1, thereby letting him run to B4, B5, B0, B1, and B2; they're back where they started, yet she's made no progress.)
- The existence of this problem, called **starvation**, can also be detected via bounded model checking. The basic idea is that starvation occurs if and only if there is a loop of transitions

$$X_0 \rightarrow X_1 \rightarrow \cdots \rightarrow X_p \rightarrow X_{p+1} \rightarrow \cdots \rightarrow X_r = X_p$$

FINAL SOLUTION

To mitigate the issues of deadlock and starvation, we introduced a new variable, l , along with a corresponding statement associated with it.

- Mutual Exclusion: Both processes use the variables a , b , and l to coordinate access to their critical sections.
- Deadlock Prevention: By using conditions based on l , Alice and Bob ensure that neither is permanently blocked from accessing the critical section.
- Starvation Avoidance: The protocol allows each process to eventually enter the critical section without indefinitely waiting for the other.

PROBLEM STATEMENT

A0. Maybe go to A1.
A1. Set $a \leftarrow 1$, go to A2.
A2. Set $l \leftarrow 0$, go to A3.
A3. If b go to A4, else to A5.
A4. If l go to A5, else to A3.
A5. Critical, go to A6.
A6. Set $a \leftarrow 0$, go to A0.

B0. Maybe go to B1.
B1. Set $b \leftarrow 1$, go to B2.
B2. Set $l \leftarrow 1$, go to B3.
B3. If a go to B4, else to B5. (49)
B4. If l go to B3, else to B5.
B5. Critical, go to B6.
B6. Set $b \leftarrow 0$, go to B0.

- We used the same idea from the first example to convert the above statements into clauses and check their satisfiability using Lingeling software.

FINAL CLAUSES

We used the same idea from the first example to convert the above statements into clauses and check their satisfiability using Lingeling software.

Clauses explanation Type 1

$(\bar{A}0_t \vee \bar{A}1_t)$	$(\bar{B}0_t \vee \bar{B}1_t)$
$(\bar{A}0_t \vee \bar{A}2_t)$	$(\bar{B}0_t \vee \bar{B}2_t)$
$\vdots$	$\vdots$
$(\bar{A}0_t \vee \bar{A}6_t)$	$(\bar{B}0_t \vee \bar{B}6_t)$
$(\bar{A}1_t \vee \bar{A}2_t)$	$(\bar{B}1_t \vee \bar{B}2_t)$
$\vdots$	$\vdots$
$(\bar{A}1_t \vee \bar{A}6_t)$	$(\bar{B}1_t \vee \bar{B}6_t)$
$\vdots$	$\vdots$
$(\bar{A}5_t \vee \bar{A}6_t)$	$(\bar{B}5_t \vee \bar{B}6_t)$

- These clauses state that neither Alice nor Bob should be present in more than one clause at the same time.
- The total number of clauses for type 1 is 42. 21 for Alice and 21 for Bob, respectively.

Type 2 clauses

$$(\alpha_t \vee \overline{A0_t} \vee A0_{t+1})$$

$$(\overline{\alpha_t} \vee \overline{B0_t} \vee B0_{t+1})$$

$$(\alpha_t \vee \overline{A1_t} \vee A1_{t+1})$$

$$(\overline{\alpha_t} \vee \overline{B1_t} \vee B1_{t+1})$$

$$\vdots$$
$$\vdots$$

$$(\alpha_t \vee \overline{A6_t} \vee A6_{t+1})$$

$$(\overline{\alpha_t} \vee \overline{B6_t} \vee B6_{t+1})$$

- These clauses state that if either Alice or Bob is running, the other must wait in its current state for time $t + 1$.
- The total number of clauses in type 2 is 14. 7 for Alice and 7 for Bob, respectively.

Type 3 clauses

$$1. (\bar{\alpha}_t \vee \bar{A0}_t \vee A0_{t+1} \vee A1_{t+1})$$

$$2. (\bar{\alpha}_t \vee \bar{A1}_t \vee A2_{t+1})$$

$$3. (\bar{\alpha}_t \vee \bar{A1}_t \vee a_{t+1})$$

$$4. (\bar{\alpha}_t \vee \bar{A2}_t \vee A3_{t+1})$$

$$5. (\bar{\alpha}_t \vee \bar{A3}_t \vee \bar{b}_t \vee A4_{t+1})$$

$$6. (\bar{\alpha}_t \vee \bar{A3}_t \vee b_t \vee A5_{t+1})$$

$$7. (\bar{\alpha}_t \vee \bar{A2}_t \vee \bar{l}_{t+1})$$

$$8. (\bar{\alpha}_t \vee \bar{A4}_t \vee \bar{l}_t \vee A5_{t+1})$$

$$9. (\bar{\alpha}_t \vee \bar{A4}_t \vee l_t \vee A3_{t+1})$$

$$10. (\bar{\alpha}_t \vee \bar{A5}_t \vee A6_{t+1})$$

$$11. (\bar{\alpha}_t \vee \bar{A6}_t \vee A0_{t+1})$$

$$12. (\bar{\alpha}_t \vee \bar{A6}_t \vee \bar{a}_{t+1})$$

similarly for Bⁱ
clauses replace

$\bar{\alpha}_t$ by α_t

b_t by a_t

$\bar{b}_t$ by $\bar{a}_t$

$\bar{l}_t$ by l_t

- The first clause states: "If Alice is running and is in state A0 at time t, and she is not in A0 at time t+1, then she must be in A1 at time t+1."
- The second clause states: "If Alice is running and is in state A1 at time t, then she must be in state A2 at time t+1."

- The third clause states that If Alice is running and is in state A1 at time t , then the value of a should be 1 at time $t+1$.
 - Similar to second clause the fourth clause have same explanation.
- The fifth clause states that if Alice is executing and, at time t , is in state A3 with the variable $b = 1$, then Alice should transition to state A4 at time $t+1$.
 - The sixth clause states that if Alice is executing and, at time t , is in state A3 with the variable $b = 0$, then Alice should transition to state A5 at time $t+1$.
 - The seventh clause states that if Alice is executing and, at time t , is in state A2, then the variable $l = 0$ at time $t+1$.
 - The eighth and ninth clauses are similar to fifth and sixth clauses respectively.
 - The other clauses for Alice follow a similar explanation.

- Bob's clauses follow the same explanation style as Alice's. Each clause for Bob specifies a condition regarding his current state, the time t , and the values of relevant variables. Based on these conditions, the clauses determine Bob's next state at time $t+1$, ensuring a consistent and predictable state transition similar to Alice's.
- The total number of clauses for type 3 is 24. 12 for Alice and 12 for Bob, respectively.

Type 4 clauses

$$\begin{aligned}\Phi(X) = & (A0 \vee A1 \vee A2 \vee A3 \vee A4 \vee A5 \vee A6) \wedge (B0 \vee B1 \vee B2 \vee B3 \vee B4 \vee B5 \vee B6) \\ & \wedge (\overline{A0} \vee \bar{a}) \wedge (\overline{A1} \vee \bar{a}) \wedge (\overline{A2} \vee a) \wedge (\overline{A3} \vee a) \wedge (\overline{A4} \vee a) \wedge (\overline{A5} \vee a) \wedge (\overline{A6} \vee a) \\ & \wedge (\overline{B0} \vee \bar{b}) \wedge (\overline{B1} \vee \bar{b}) \wedge (\overline{B2} \vee b) \wedge (\overline{B3} \vee b) \wedge (\overline{B4} \vee b) \wedge (\overline{B5} \vee b) \wedge (\overline{B6} \vee b). \quad (50)\end{aligned}$$

- The first two clauses state that, at any given instant, either Alice or Bob must be in at least one state.
- This ensures that both entities are consistently active within the system, with at least one of them occupying a defined state at any moment in time.
- The remaining clauses provide the values of variables a and b when Alice and Bob are in states Ax and Bx , respectively.
- These clauses outline specific conditions under which a and b take on certain values, depending on Alice's and Bob's current states.
- This allows us to determine the state-dependent values of a and b during Alice's and Bob's execution.
- The total number of clauses for type 4 is 16. 8 for Alice and 8 for Bob, respectively.

Type 5 clauses

$$(\bar{\alpha}_t \vee \bar{A4}_t \vee \bar{B5}_t \vee \bar{l}_t)$$

$$(\alpha_t \vee \bar{B4}_t \vee \bar{A5}_t \vee l_t)$$

$$(\bar{\alpha}_t \vee \bar{A3}_t \vee \bar{B5}_t \vee b_t)$$

$$(\alpha_t \vee \bar{B3}_t \vee \bar{A5}_t \vee b_t)$$

- The first clause states that if Alice is executing and is in state A4 at time t , and Bob is in state B5 at the same time, then the value of the variable l must be 0 at time t . This condition enforces a specific value for l based on the simultaneous states of Alice and Bob.
 - Similar explanation for other clauses.
- The total number of clauses for type 5 is 4. 2 for Alice and 2 for Bob, respectively.

Type 6 clauses

$$(\alpha_t \vee \bar{A}0_t \vee \bar{B}5_t \vee \bar{a}_{t+1})$$

$$(\alpha_t \vee \bar{A}1_t \vee \bar{B}5_t \vee \bar{a}_{t+1})$$

$$(\bar{\alpha}_t \vee \bar{A}2_t \vee \bar{B}5_t \vee \bar{l}_{t+1})$$

$$(\bar{\alpha}_t \vee \bar{A}3_t \vee \bar{B}5_t \vee \bar{l}_{t+1})$$

$$(\bar{\alpha}_t \vee \bar{A}4_t \vee \bar{B}5_t \vee \bar{l}_{t+1})$$

Similarly for B's
clauses replace
 α_t by $\bar{\alpha}_t$
 a_{t+1} by b_{t+1}
 l_{t+1} by $\bar{l}_{t+1}$

- These clauses are introduced to ensure that the variables l and a remain 0 at time $t+1$ when Alice or Bob are in a critical state. This is done to satisfy the type 4 clauses.
- By enforcing these clauses, we ensure that the values of l and a are properly controlled, particularly during critical states, so that the system behaves as expected according to the type 4 conditions.
- The total number of clauses for type 6 is 10. 5 for Alice and 5 for Bob, respectively.

Type-7 clauses

$$A5_r \wedge B5_r .$$

- The above unit clauses states that both Alice and Bob are entering the critical section at time $t = r$. If the output is unsatisfied, it indicates that Alice and Bob are not entering the critical section using this protocol.
- The total number of clauses for type 7 is 2. 1 for Alice and 1 for Bob, respectively.

By using invariants, we can confidently conclude that if the output is unsatisfiable for both path lengths of 1 and 2, then the entire problem is unsatisfiable.

So in the slides following this we have just considered for $\text{pathlength}=1$ and $\text{pathlength}=2$.

Summary for path-1

- Here path-1 means $r=1$
- The possible state from which execution can begin is $t=0$, while the possible states that can be reached are $t=0, t=1$.

So total number of clauses for path-1 include -

- 1) type-1 clauses at $t=0$ and $t=1 \Rightarrow 42+42=84$.
- 2) type-2 clauses at $t=0 \Rightarrow 14$.
- 3) type-3 clauses at $t=0 \Rightarrow 24$.
- 4) type-4 clauses at $t=0 \Rightarrow 16$.
- 5) type-5 clauses at $t=0 \Rightarrow 4$.
- 6) type-6 clauses at $t=0 \Rightarrow 10$.
- 7) type-7 clauses at $t=1 \Rightarrow 2$.

Total number clauses = 154.

Total number of variables = 35.

Summary for path-2

- Here path-2 means $r=2$
- The possible states from which execution can begin are $t=0$ and $t=1$, while the possible states that can be reached are $t=0$, $t=1$, and $t=2$.

So total number of clauses for path-2 include -

1) type-1 clauses at $t=0$, $t=1$ and $t=2 \Rightarrow 42+42+42 = 126$.

2) type-2 clauses at $t=0$ and $t=1 \Rightarrow 14+14 = 28$.

3) type-3 clauses at $t=0$ and $t=1 \Rightarrow 24+24 = 48$.

4) type-4 clauses at $t=0$ and $t=1 \Rightarrow 16+16 = 32$.

5) type-5 clauses at $t=0$ and $t=1 \Rightarrow 4+4 = 8$.

6) type-6 clauses at $t=0$ and $t=1 \Rightarrow 10+10 = 20$.

7) type-7 clauses at $t=2 \Rightarrow 2$.

Total number clauses = 264.

Total number of variables = 52.

Output for path-1

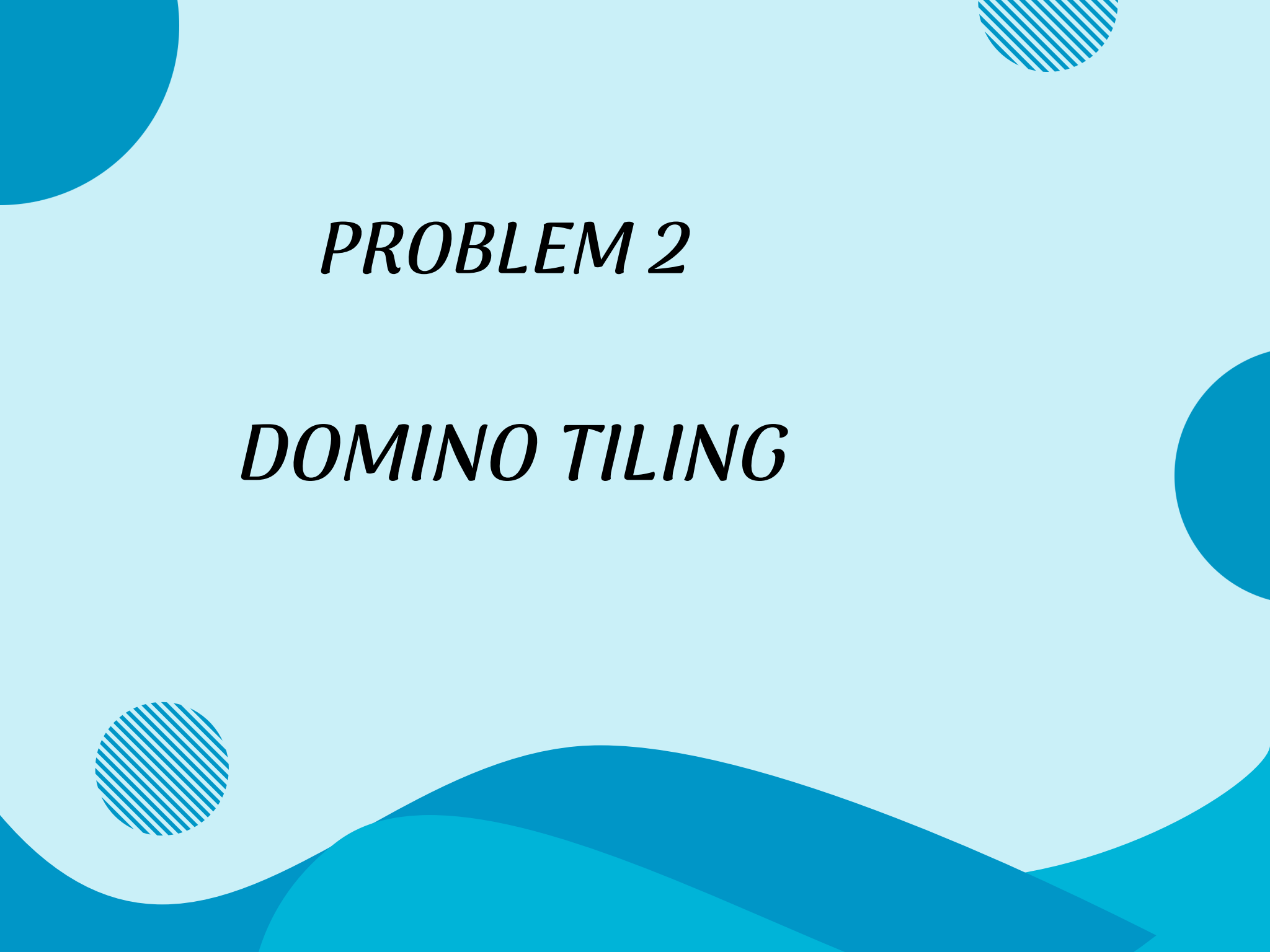
```
(base) kovid@Kovids-MacBook-Air lingeling-master % ./lingeling path1.cnf
c Lingeling SAT Solver
c
c Version 1.0.0
c
c Copyright (C) 2010-2016 Armin Biere JKU Linz Austria.
c All rights reserved.
c
c released Thu Oct 3 10:02:02 IST 2024
c compiled Thu Oct 3 10:02:02 IST 2024
c
c Apple clang version 14.0.0 (clang-1400.0.29.202)
c -W -Wall -O3 -DNLGLOG -DNDEBUG -DNCHKSOL -DNLGLDRUPLIG -DNLGLYALSAT
c -DNLGLFILES -DNLGLDEMA
c Darwin Kovids-MacBook-Air.local 22.5.0 arm64
c
c
c seconds      irredundant      redundant clauses  glue  iterations"  MB  stability
c              variables clauses conflicts  large ternary binary  jlevel  jlevel'  agility  tlevel
c
c S   0.0       21      152        0        0      0      0      0      0      0      0      0      0
c
c seconds      irredundant      redundant clauses  glue  iterations"  MB  stability
c              variables clauses conflicts  large ternary binary  jlevel  jlevel'  agility  tlevel
c
c s UNSATISFIABLE
c
c   0.001  91% simplifying
c   0.000   0% search
c =====
c   0.001 100% all
c
c           0 conflicts,           0.0 confs/sec
c           0 ternaries,          0.0 confs/ternary
c           0 binaries,           0.0 confs/binary
c           0 iterations,          0.0 confs/iteration
c
c           0 reductions,          0.0 redus/sec,      0.0 confs/reduction
c           0 restarts,            0.0 rests/sec,      0.0 confs/restart
c           0 decisions,           0.0 decis/sec,       0.0 decis/conflict
c           26 propagations,       31862.7 props/sec,  0.0 props/decision
c
c 0.0 seconds, 0.0 MB
```

- Here output is **UNSATISFIABLE** means our encoding for path-1 is correct.

Output for path-2

```
(base) kovid@Kovids-MacBook-Air lingeling-master % ./lingeling path2.cnf
c Lingeling SAT Solver
c
c Version 1.0.0
c
c Copyright (C) 2010-2016 Armin Biere JKU Linz Austria.
c All rights reserved.
c
c released Thu Oct  3 10:02:02 IST 2024
c compiled Thu Oct  3 10:02:02 IST 2024
c
c Apple clang version 14.0.0 (clang-1400.0.29.202)
c -W -Wall -O3 -DNLGLLOG -DNDEBUG -DNCHKSOL -DNLGLDRUPLIG -DNLGLYALSAT
c -DNLGLFILES -DNLGLDEMA
c Darwin Kovids-MacBook-Air.local 22.5.0 arm64
c
c
c seconds      irredundant      redundant clauses  glue  iterations"  MB  stability
c      variables clauses conflicts large ternary binary  jlevel  jlevel' agility  tlevel
c
c S   0.0       39       262       0       0       0       0       0       0       0       0       0       0
c
c seconds      irredundant      redundant clauses  glue  iterations"  MB  stability
c      variables clauses conflicts large ternary binary  jlevel  jlevel' agility  tlevel
c
c UNSATISFIABLE
c
c   0.001  94% simplifying
c   0.000   0% search
c =====
c   0.001 100% all
c
c      0 conflicts,      0.0 confs/sec
c      0 ternaries,    0.0 confs/ternary
c      0 binaries,     0.0 confs/binary
c      0 iterations,   0.0 confs/iteration
c
c      0 reductions,   0.0 redus/sec,   0.0 confs/reduction
c      0 restarts,     0.0 rests/sec,   0.0 confs/restart
c      0 decisions,    0.0 decis/sec,   0.0 decis/conflict
c      37 propagations, 27987.9 props/sec, 0.0 props/decision
c
c 0.0 seconds, 0.0 MB
```

- Here output is **UNSATISFIABLE** means our encoding for path-2 is correct.

The background is a light blue color. It features several decorative elements: a large solid blue circle in the top-left corner, a smaller circle with blue diagonal stripes in the top-right corner, a solid blue circle on the right edge, and a circle with blue diagonal stripes in the bottom-left corner. At the bottom, there are large, overlapping wavy shapes in two shades of blue.

PROBLEM 2

DOMINO TILING

Problem Overview

The mutilated chessboard problem involves covering a chessboard where one or more squares are removed with a set number of dominoes.

Each domino covers exactly two squares, so placing 71 dominoes covers 142 squares.

In this case, we are given a 12×12 board (144 squares total).

We need to place 71 dominoes to cover 142 squares, meaning 2 squares are removed from the board.

Cases of Color Removal for the Mutilated Chessboard Problem



In a 12×12 chessboard, each domino must cover one black and one white square

Case 1: Same Color Removal

- Removing two squares of the same color (either both black or both white) creates an imbalance between black and white squares.
- **Result:** The problem is unsatisfiable (UNSAT) because dominos cannot fully cover the board due to the color mismatch.

Case 2: Different Color Removal

- Removing one black square and one white square maintains an equal number of black and white squares.
- **Result:** The problem is satisfiable (SAT), as the board can be completely covered by 71 dominos while maintaining the required color pairing

Encoding Each Square (i,j) with SAT Variables

In this SAT encoding for a grid, each square (i,j) has four variables to represent its coverage status.

The variables denote directions relative to the square:

- $x_{i,j,1}$: Indicates the cell at position (i,j) is covered by the left side of a domino placed horizontally.
- $x_{i,j,-1}$: Indicates the cell at position (i,j) is covered by the right side of a domino placed horizontally.
- $x_{i,j,2}$: Indicates the cell at position (i,j) is covered by the top side of a domino placed vertically.
- $x_{i,j,-2}$: Indicates the cell at position (i,j) is covered by the bottom side of a domino placed vertically.

Total Variables = $142 \times 4 = 568$

SAT Encoding for Domino Placement Constraints

1. Each cell (i, j) on the grid can be covered by only one specific side of a domino, ensuring no overlaps or multiple coverings.
2. **Mutual Exclusion Clauses:**
 - $(\neg x_{i,j,1} \vee \neg x_{i,j,-1})$: If (i, j) is covered by the left side, it cannot also be covered by the right side.
 - $(\neg x_{i,j,1} \vee \neg x_{i,j,2})$: If (i, j) is covered by the left side, it cannot be covered by the top side.
 - $(\neg x_{i,j,1} \vee \neg x_{i,j,-2})$: If (i, j) is covered by the left side, it cannot be covered by the bottom side.
 - $(\neg x_{i,j,2} \vee \neg x_{i,j,-2})$: If (i, j) is covered by the top side, it cannot also be covered by the bottom side.
 - Similar clauses ensure that if (i, j) is covered by the top side, it cannot be covered by the left or right side.
3. These clauses together guarantee that each cell is uniquely covered, aligning with the domino tiling constraints.

These clauses in the SAT encoding ensure that each domino placement is consistent with the required grid constraints, meaning that each domino covers two adjacent cells. The clauses are:

1. $(\neg x_{i,j,1} \vee x_{i,j+1,-1})$: If cell (i, j) is covered by the left side of a horizontally placed domino, then the adjacent cell to its right $(i, j + 1)$ must be covered by the right side of the same domino.
2. $(\neg x_{i,j,-1} \vee x_{i,j-1,1})$: If cell (i, j) is covered by the right side of a horizontally placed domino, then the adjacent cell to its left $(i, j - 1)$ must be covered by the left side of the same domino.
3. $(\neg x_{i,j,2} \vee x_{i+1,j,-2})$: If cell (i, j) is covered by the top side of a vertically placed domino, then the cell directly below it $(i + 1, j)$ must be covered by the bottom side of the same domino.
4. $(\neg x_{i,j,-2} \vee x_{i-1,j,2})$: If cell (i, j) is covered by the bottom side of a vertically placed domino, then the cell directly above it $(i - 1, j)$ must be covered by the top side of the same domino.

Purpose: These clauses ensure that dominoes are placed in pairs across adjacent cells, either horizontally or vertically, without overlapping or leaving gaps between paired cells.

The clause $x_{i,j,1} \vee x_{i,j,-1} \vee x_{i,j,2} \vee x_{i,j,-2}$ ensures that each cell (i, j) is covered by one side of a domino. This allows each cell to be part of a domino either horizontally or vertically, ensuring complete coverage of the grid. It prevents any cell from being left uncovered.

CODE TO GENERATE A CNF (CONJUNCTIVE NORMAL FORM) FILE FOR USE WITH A SAT SOLVER (LINGELING)

```
#include <bits/stdc++.h>
#define inti int
using namespace std;
int main() {
    vector<vector<inti>> clauses;
    inti n;
    cin>>n;
    inti r1,c1;
    inti r2,c2;
    cin>>r1>>c1;//to be removed
    cin>>r2>>c2;//to be removed
    inti variable=1;
    for(inti row=1;row<=n;row++){
        for(inti column=1;column<=n;column++){
            if(row==r1 && column==c1){
                continue;
            }
            if(row==r2 && column==c2){
                continue;
            }
            vector<vector<inti>> cur;
            inti a=variable++;
            inti b=variable++;
            inti c=variable++;
            inti d=variable++;
            clauses.push_back({a,b,c,d});
            clauses.push_back({-a,-b});
            clauses.push_back({-a,-c});
            clauses.push_back({-a,-d});
            clauses.push_back({-b,-c});
            clauses.push_back({-b,-d});
            clauses.push_back({-c,-d});
        }
        inti newrow=row; inti newcolumn=column-1;
        if(newrow<1 || newrow>n || newcolumn<1 || newcolumn>n
        || (newrow==r1 && newcolumn==c1) || (newrow==r2 && newcolumn==c2)){
            clauses.push_back({-b});
        }
        else{
            clauses.push_back({-b,b-5});
        }
        newrow=row; newcolumn=column+1;
        if(newrow<1 || newrow>n || newcolumn<1 || newcolumn>n
        || (newrow==r1 && newcolumn==c1) || (newrow==r2 && newcolumn==c2)){
            clauses.push_back({-a});
        }
    }
}
```



```

        }
        clauses.push_back({-a,a+5});
    }
    newrow=row+1; newcolumn=column;
    inti tobeadded=n*4;
    if(row==r1 && column<c1){
        tobeadded-=4;
    }
    if(row==r2 && column<c2){
        tobeadded-=4;
    }
    if(newrow==r1 && newcolumn>c1){
        tobeadded-=4;
    }
    if(newrow==r2 && newcolumn>c2){
        tobeadded-=4;
    }
    if(newrow<1 || newrow>n || newcolumn<1 || newcolumn>n
    || (newrow==r1 && newcolumn==c1) || (newrow==r2 && newcolumn==c2)){
        clauses.push_back({-c});
    }
    else{
        clauses.push_back({-c,c+1+tobeadded});
    }
    newrow=row-1; newcolumn=column;
    inti tobesubtracted=n*4;
    if(row==r1 && column>c1){
        tobesubtracted-=4;
    }
    if(row==r2 && column>c2){
        tobesubtracted-=4;
    }
    if(newrow==r1 && newcolumn<c1){
        tobesubtracted-=4;
    }
    if(newrow==r2 && newcolumn<c2){
        tobesubtracted-=4;
    }
    if(newrow<1 || newrow>n || newcolumn<1 || newcolumn>n
    || (newrow==r1 && newcolumn==c1) || (newrow==r2 && newcolumn==c2)){
        clauses.push_back({-d});
    }
}

```

```

        }
        clauses.push_back({-d,d-1-tobesubtracted});
    }
}

cout<<"p cnf " <<variable-1<<" " <<clauses.size()<<endl;
for(inti i=0;i<clauses.size();i++){
    inti cursize=clauses[i].size();
    for(inti j=0;j<cursize;j++){
        cout<<clauses[i][j]<<" ";
    }
    cout<<0<<endl;
}
}

```

OVERVIEW OF HOW THE CODE WORKS

Input

- n : The size of the chessboard (in this case, 12 for a 12x12 board).
- $(r1, c1)$ and $(r2, c2)$: Coordinates of two cells that are removed from the board.

Variable Representation:

- Each cell (i, j) on the board is represented by four variables:
- $X_{i,j,1}$ (a): Indicates the cell is covered by the left side of a horizontally placed domino.
- $X_{i,j,-1}$ (b): Indicates the cell is covered by the right side of a horizontally placed domino.
- $X_{i,j,2}$ (c): Indicates the cell is covered by the top side of a vertically placed domino.
- $X_{i,j,-2}$ (d): Indicates the cell is covered by the bottom side of a vertically placed domino.
- The variable counter keeps track of these four variables for each cell.

Single Domino Coverage:

Each cell must be covered by exactly one domino. The code enforces this by creating clauses:

- $\{a, b, c, d\}$ ensures that one of these options must be true (at least one direction covers the cell).
- $\{-a, -b\}$, $\{-a, -c\}$, $\{-a, -d\}$, $\{-b, -c\}$, $\{-b, -d\}$, and $\{-c, -d\}$ ensure that no two directions simultaneously cover the cell

Neighbor to the Right :

- `newcolumn = column + 1`: Sets the potential right neighbor's column.

Boundary Check:

- `if(newcolumn > n || newrow > n || (newrow == r1 && newcolumn == c1) || (newrow == r2 && newcolumn == c2))`: Checks if this right neighbor is outside the board bounds or one of the removed cells.
- If true, `clauses.push_back({-a})` adds a clause $\{-a\}$, indicating that if cell (i, j) has a (left side of a horizontal domino) set, it cannot find a valid right neighbor to pair with.

Valid Right Neighbor:

- If a valid neighbor exists, `clauses.push_back({-a, a+5})` links a to its neighboring variable $a + 5$, enforcing that if the current cell is covered by the left side of a horizontal domino, the right neighbor must be covered by the right side of the same domino.

Neighbor to the Left :

- Set Left Neighbor:
- $\text{newcolumn} = \text{column} - 1$ sets the potential left neighbor's column index by moving one cell to the left.

Boundary Check:

- The condition $\text{if } (\text{newcolumn} < 1 \parallel \text{newrow} < 1 \parallel (\text{newrow} == r1 \ \&\& \ \text{newcolumn} == c1) \parallel (\text{newrow} == r2 \ \&\& \ \text{newcolumn} == c2))$ checks if this left neighbor is outside the grid bounds (i.e., $\text{newcolumn} < 1$) or is one of the removed cells at $(r1, c1)$ or $(r2, c2)$.
- If true, `clauses.push_back({-b})` adds a clause $\{-b\}$, indicating that if cell (i, j) has b (right side of a horizontal domino) set, it cannot find a valid left neighbor to pair with.

Valid Left Neighbor:

- If a valid left neighbor exists, `clauses.push_back({-b, b-5})` links b (right side of the domino) to the neighboring variable $b - 5$, representing the left side of the horizontal domino.

Neighbor Below :

- $\text{newrow} = \text{row} + 1$: Sets the potential lower neighbor's row.

Boundary Check:

- $\text{if}(\text{newrow} > n \parallel (\text{newrow} == r1 \ \&\& \ \text{newcolumn} == c1) \parallel (\text{newrow} == r2 \ \&\& \ \text{newcolumn} == c2))$.
Ensures the potential lower neighbor is within bounds and not a removed cell.
- If true, $\text{clauses.push_back}\{-c\}$ adds a clause $\{-c\}$, meaning if cell (i, j) has c (top side of a vertical domino) set, there is no valid bottom neighbor to pair with.

Valid Lower Neighbor:

- If a valid lower neighbor exists, $\text{clauses.push_back}\{-c, c + 1 + \text{tobeadded}\}$ links c to the adjusted neighbor variable, enforcing the pairing.

Calculating tobeadded

- Since each row contains n cells, and each cell has four variables:
One row of cells has $n * 4$ variables.

Thus:

$\text{tobeadded} = n * 4$ ensures that we jump to the correct variable index for the row below.

Adjustments to tobeadded for Removed Cells

- If there are removed cells in certain positions (relative to the current cell), they might affect the continuity of variable indexing. For instance:
- If a cell is removed to the left or above the current cell, it affects how we count variables in the row below.
- To correct for this, the code decreases tobeadded by 4 whenever such a removed cell exists.

Neighbor Above :

`newrow = row - 1`: Sets the potential upper neighbor.

Boundary Check:

- If the upper neighbor is out of bounds or a removed cell, `clauses.push_back({-d})` adds a clause `{-d}`, indicating that the current cell cannot be covered by the bottom side of a vertical domino.

Valid Upper Neighbor:

- If a valid neighbor exists, `clauses.push_back({-d, d - 1 - tobesubtracted})` links `d` to the corresponding neighbor, ensuring the domino covers both cells.

Calculating tobesubtracted

- To ensure we reach the correct variable index of the upper neighbor, `tobesubtracted` is set to:
- `tobesubtracted = n * 4`, which shifts the variable index back by one row.

Adjustments to tobesubtracted for Removed Cells:

- If removed cells are near the current cell, they can affect variable indexing for the row above. To address this, `tobesubtracted` is decreased by 4 (the number of variables per cell) when:
 - A cell is removed to the right or below the current cell.
 - A cell is removed to the left or above the cell directly above.

CASE 1 :WHEN ONE WHITE AND ONE BLACK CELL IS REMOVED

INPUT:

12

1 2

9 9

```
c
s SATISFIABLE
v -1 -2 3 -4 -5 -6 7 -8 9 -10 -11 -12 -13 14 -15 -16 -17 -18 19 -20 21 -22 -23
v -24 -25 26 -27 -28 -29 -30 31 -32 33 -34 -35 -36 -37 38 -39 -40 -41 -42 43 -44
v -45 -46 47 -48 -49 -50 -51 52 -53 -54 -55 56 57 -58 -59 -60 -61 62 -63 -64 -65
v -66 -67 68 69 -70 -71 -72 -73 74 -75 -76 -77 -78 -79 80 -81 -82 83 -84 -85 -86
v 87 -88 -89 -90 -91 92 -93 -94 -95 96 -97 -98 99 -100 101 -102 -103 -104 -105
v 106 -107 -108 109 -110 -111 -112 -113 114 -115 -116 -117 -118 119 -120 121
v -122 -123 -124 -125 126 -127 -128 -129 -130 -131 132 -133 -134 -135 136 -137
v -138 139 -140 -141 -142 143 -144 -145 -146 -147 148 -149 -150 151 -152 -153
v -154 155 -156 157 -158 -159 -160 -161 162 -163 -164 -165 -166 -167 168 169
v -170 -171 -172 -173 174 -175 -176 -177 -178 179 -180 -181 -182 183 -184 -185
v -186 -187 188 -189 -190 -191 192 -193 -194 195 -196 -197 -198 -199 200 -201
v -202 -203 204 205 -206 -207 -208 -209 210 -211 -212 213 -214 -215 -216 -217
v 218 -219 -220 -221 -222 223 -224 -225 -226 -227 228 -229 -230 -231 232 -233
v -234 235 -236 -237 -238 239 -240 -241 -242 -243 244 245 -246 -247 -248 -249
v 250 -251 -252 253 -254 -255 -256 -257 258 -259 -260 -261 -262 263 -264 -265
v -266 267 -268 -269 -270 -271 272 273 -274 -275 -276 -277 278 -279 -280 -281
v -282 -283 284 -285 -286 -287 288 289 -290 -291 -292 -293 294 -295 -296 297
v -298 -299 -300 -301 302 -303 -304 -305 -306 307 -308 -309 -310 -311 312 -313
v -314 -315 316 -317 -318 319 -320 -321 -322 323 -324 -325 -326 327 -328 329
v -330 -331 -332 -333 334 -335 -336 337 -338 -339 -340 -341 342 -343 -344 345
v -346 -347 -348 -349 350 -351 -352 -353 -354 -355 356 -357 -358 359 -360 -361
v -362 363 -364 -365 -366 -367 368 -369 -370 -371 372 -373 -374 -375 376 -377
v -378 379 -380 -381 -382 383 -384 -385 -386 387 -388 389 -390 -391 -392 -393
v 394 -395 -396 -397 -398 399 -400 -401 -402 403 -404 -405 -406 -407 408 -409
v -410 -411 412 413 -414 -415 -416 -417 418 -419 -420 -421 -422 423 424 425
v -426 -427 428 -429 -430 -431 432 -433 -434 -435 436 437 -438 -439 -440 441
v 442 -443 -444 -445 -446 -447 448 -449 450 -451 452 -453 -454 455 456 457
v -458 459 -460 461 -462 -463 -464 -465 466 -467 -468 -469 -470 -471 472 473
v -474 -475 -476 -477 478 -479 -480 481 -482 -483 484 -485 486 -487 -488 489
v -490 -491 -492 -493 494 -495 -496 -497 -498 499 -500 -501 -502 -503 504 -505
v -506 -507 508 -509 -510 511 -512 -513 -514 515 -516 -517 -518 519 -520 521
v -522 -523 -524 -525 526 -527 -528 529 -530 -531 -532 -533 534 -535 -536 537
v -538 -539 -540 -541 542 -543 -544 -545 -546 -547 548 549 -550 -551 -552 -553
v 554 -555 -556 -557 -558 -559 560 -561 -562 -563 564 -565 -566 -567 568 0
c
```

CASE 2 :WHEN TWO WHITE OR TWO BLACK CELLS REMOVED

Input: 12
 12 12
 9 3

```
UNSATISFIABLE
0.000 9% simplifying
0.000 0% search
=====
0.000 100% all

0 conflicts,          0.0 confs/sec
0 ternaries,         0.0 confs/ternary
0 binaries,          0.0 confs/binary
0 iterations,        0.0 confs/iteration

0 reductions,         0.0 redus/sec,    0.0 confs/reduction
0 restarts,           0.0 rests/sec,    0.0 confs/restart
0 decisions,          0.0 decs/sec,     0.0 decs/conflict
36 propagations,     514285.7 props/sec,    0.0 props/decision
```

Bonus question

*Impact of the elim Parameter on Lingeling
SAT Solver Performance*

Optimizing with Elim Parameter

- **Goal:** Test performance with different values of the --elim parameter in Lingeling
- **Objective:** Identify how varying the degree of variable elimination impacts solving time.
- **Method:** Cactus plot analysis across various --elim values

What is the "elim" Parameter?

Elimination in SAT solvers like Lingeling is a preprocessing step where certain variables or clauses are removed or simplified before the main solving begins.

The elim parameter determines the extent of this elimination process, allowing the solver to decide which variables or clauses should be removed or ignored to reduce the complexity of the problem.

Impact on Performance:

- **Eliminating Redundant Variables:** By removing variables that do not contribute significantly to the solution, the solver can focus on the most important parts of the formula, potentially reducing the solving time.

- **Simplification:** The elim parameter helps to simplify the problem by eliminating variables or clauses that would otherwise make the problem more complex to solve, leading to faster solutions.
- **Trade-Off:** While elimination can speed up the solver, too much elimination could lead to the removal of crucial information, which might negatively affect the solver's ability to find a solution. Hence, the right value of elim needs to be chosen carefully.

Common Usage:

- elim=0: Often means no elimination is performed, or minimal elimination is applied.
- elim=1, 2, 3, ...: Increasing values of elim may involve progressively more aggressive elimination strategies, leading to potentially faster solving times for certain problems.

Python Script for Lingeling Benchmark and Cactus Plot Generation

```
import subprocess
import concurrent.futures
import matplotlib.pyplot as plt

# Paths and configurations
cnf_directory = "/Users/kovid/Downloads/satsolver"
cnf_files = sorted([os.path.join(cnf_directory, f) for f in os.listdir(cnf_directory) if f.endswith('.cnf')][:20]) # Sort CNF files
elim_values = [0, 5, 10] # Adjust values for --elim parameter
timeout = 900 # 900-second timeout per instance

# Function to run Lingeling with specified elim parameter and capture runtime
def run_lingeling(cnf_file, elim_value):
    command = ['./lingeling', f'--elim={elim_value}', cnf_file]
    print(f"Running Lingeling on {cnf_file} with --elim={elim_value}")
    start_time = time.time()
    try:
        result = subprocess.run(command, capture_output=True, text=True, timeout=timeout)
        end_time = time.time()
        status = 'UNSAT' if 'UNSATISFIABLE' in result.stdout else 'SAT' if 'SATISFIABLE' in result.stdout else 'UNKNOWN'
        runtime = end_time - start_time
        print(f"Completed {cnf_file} in {runtime:.3f} seconds with status {status}")
        return runtime
    except subprocess.TimeoutExpired:
        print(f"Timed out on {cnf_file}")
        return timeout

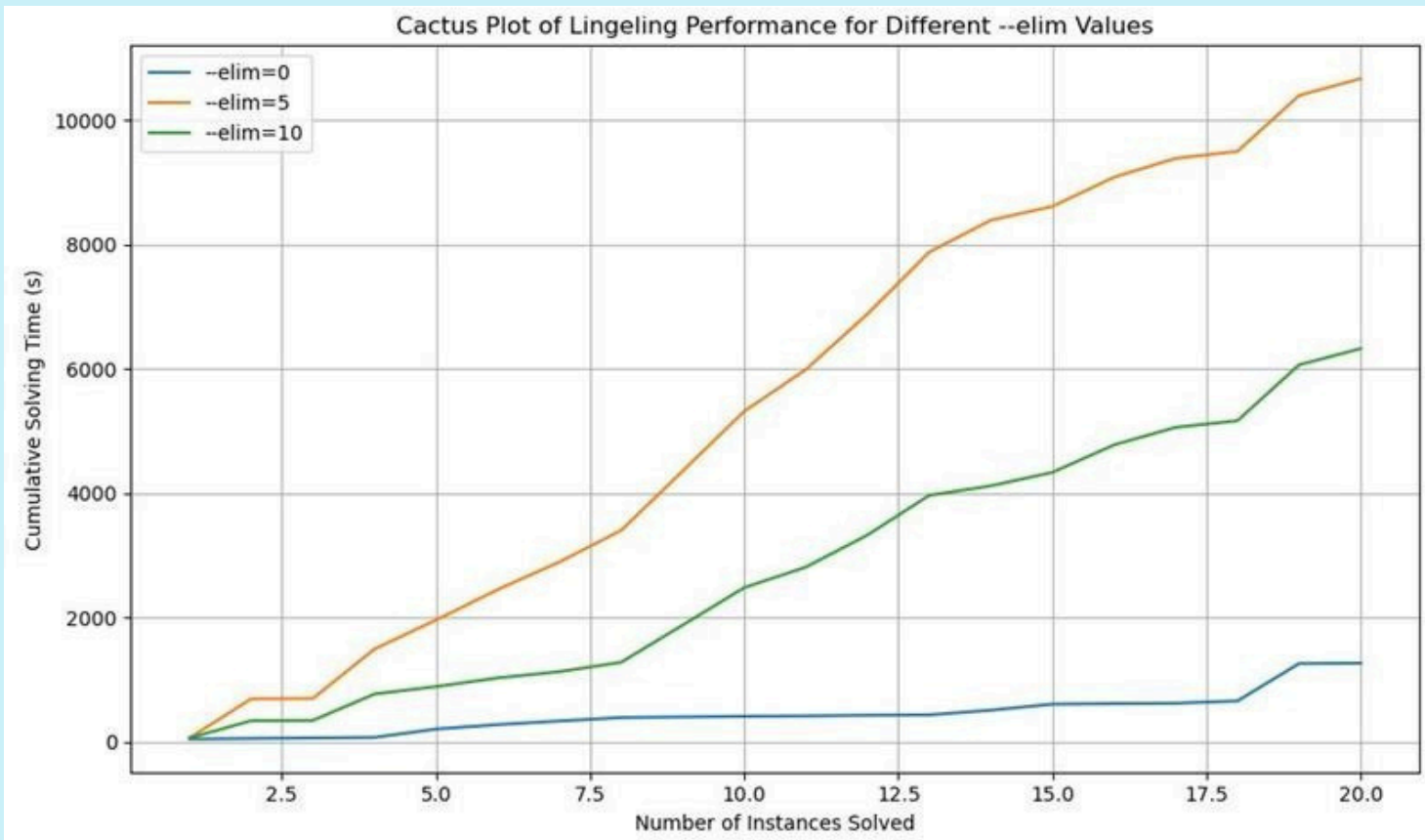
# Run benchmarks in parallel
def benchmark_elim(elim_value):
    with concurrent.futures.ThreadPoolExecutor() as executor:
        times = list(executor.map(lambda cnf_file: run_lingeling(cnf_file, elim_value), cnf_files))
    print(f"Times for --elim={elim_value}: {times}")
    return times
```

```
# Collect results for each elim value
results = {elim_value: benchmark_elim(elim_value) for elim_value in elim_values}

# Prepare data for cactus plot
cumulative_solved = {}
for elim_value, times in results.items():
    cumulative_times = []
    total_time = 0
    for time in times:
        total_time += time
        cumulative_times.append(total_time)
    cumulative_solved[elim_value] = cumulative_times

# Generate cactus plot and save as PNG
plt.figure(figsize=(10, 6))
for elim_value, cumulative in cumulative_solved.items():
    plt.plot(range(1, len(cumulative) + 1), cumulative, label=f'--elim={elim_value}')
plt.xlabel("Number of Instances Solved")
plt.ylabel("Cumulative Solving Time (s)")
plt.title("Cactus Plot of Lingeling Performance for Different --elim Values")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("cactus_plot_lingeling.png")
print("Cactus plot saved as 'cactus_plot_lingeling.png'")
```

Cactus Plot Analysis



Observations:

- **elim=0 (Blue line)** achieves the lowest cumulative solving time across all instances. It suggests that minimal preprocessing (no variable elimination) is optimal for this dataset, as it avoids overhead associated with variable elimination.
- **elim=5 (Orange line)** and **elim=10 (Green line)** show progressively higher solving times, with **elim=5** solving fewer instances within the same cumulative time range compared to **elim=0**, and **elim=10** requiring even more time

Why Does Cumulative Time Increase Then Decrease?

Initial Increase:

Higher elimination settings (elim=5 and elim=10) introduce more preprocessing steps, causing a significant initial rise in solving time. The solver spends extra time analyzing and simplifying the instances, which doesn't always reduce complexity effectively, especially in complex cases.

Later Slowdown in Time Growth:

As more instances are solved, cumulative time growth slows down.

This could be due to:

Effective Simplifications: Some instances eventually benefit from variable elimination, leading to faster solves and a leveling off of time.

Variation in Instance Complexity: After solving the most complex cases, remaining instances are often simpler, reducing time per instance.

Conclusion

- **Optimal Setting:** elim=0 delivers the best performance for this dataset, minimizing cumulative solving time by avoiding unnecessary preprocessing overhead.
- **Impact of Higher elim Values:** Increasing --elim values (5 and 10) leads to additional computational overhead due to aggressive variable elimination, resulting in slower overall solving times.
- Recommendation: For similar SAT instances, using elim=0 or lower elim values can improve solver efficiency by reducing preprocessing costs. Higher elim values may be beneficial only for datasets where variable elimination yields substantial simplifications.

The background is a light blue color. It features several abstract geometric shapes in different shades of blue. In the top left corner, there is a large solid blue circle. In the top right corner, there is a smaller circle with diagonal blue and white stripes. In the bottom left corner, there is another circle with diagonal blue and white stripes. Along the bottom edge, there are several overlapping, wavy, solid blue shapes that resemble stylized hills or waves. The text "THANK YOU!" is centered in the upper half of the image.

**THANK
YOU!**